

Briefing: DRS High-Precision User-Space Synchronization

Context: We are building a Distributed Real-Time System (DRS) application to establish a highly precise global time base across a dynamic cluster of hardware nodes (e.g., Raspberry Pis) using UDP sockets in C/C++. The nodes will communicate over mixed networks (Ethernet and Wi-Fi).

The Paradigm: Do not jump straight to generating code. You will use the "Vibe Coding" V-Model. Use this briefing as your starting input for Phase 1. Prompt your AI assistant to critique these notes, extrapolate implicit requirements (numbered F-1, NF-1, etc.), and resolve architectural open questions before synthesizing the code.

1. System Objectives

You must design a synchronization protocol that runs entirely in the user-space of a standard Linux OS (no kernel drivers). The overarching goal is to achieve a synchronization precision of $< 100 \mu\text{s}$ between the nodes.

The system must follow the KISS principle (Keep It Simple, Stupid): It should require zero configuration regarding network addresses or roles when a new node is plugged in. Nodes must discover each other autonomously.

2. DRS Engineering Concepts to Anchor

When guiding the AI to build your Architecture Anchor, ensure it accounts for these core DRS principles:

- **Global Time vs. Local Time:** A global time is an abstract notion approximated by the local physical clocks. However, forcibly stepping or constantly adjusting the local Linux system clock via system calls can destroy the monotonic properties required for other local control tasks. How can a user-space application maintain a synchronized global time without interfering with the OS hardware clock?
- **Latency Jitter & Topologies:** The latency jitter is the most important term affecting the precision of the synchronization. Wi-Fi introduces highly asymmetrical jitter (CSMA/CA). Your protocol must dynamically filter these outliers. Furthermore, consider the topology: A central master creates a single point of failure. How can the protocol adapt if a leader node suddenly suffers a crash failure?
- **Structured Error Handling:** Error handling is usually the least designed subsystem, leading to unintended behavior. You must define a structured error handling strategy: For each detectable error, determine the root cause, the suitable system-level action, and the strategy to indicate or store the error. "Just printing to stdout" is not a strategy.
- **Deployment, Integration & The Probe Effect:** A distributed system needs to be deployed and debugged across multiple nodes simultaneously. How will you distribute and deploy the software? How will you configure the nodes? During integration testing, the modification of the behavior by introducing a test probe is called the probe effect. How will you debug the synchronization accuracy and inspect node states without ruining the $< 100 \mu\text{s}$ timing constraints through your own debugging outputs?

3. Required Test Scenarios

To prove the composability and resilience of your architecture, you must define and execute the following test scenarios:

1. **Baseline:** 3 nodes on Ethernet. Measure and log the achieved precision over 10 minutes.
2. **Dynamic Discovery:** Start with 2 nodes, wait for sync, plug in a 3rd node. It must integrate seamlessly.
3. **Crash Failure Tolerance:** Pull the power plug of the node currently acting as the primary time reference. The remaining nodes must maintain synchronization.
4. **High Jitter Resilience:** Run the cluster over a saturated Wi-Fi network and evaluate the degradation.

4. Verifiability & Proof of Function (Falsifiability)

A core principle of engineering is falsifiability. If you claim your user-space protocol achieves a synchronization precision of $< 100 \mu\text{s}$, you must prove it objectively. You cannot simply trust the internal software logs or timestamps generated by your own protocol, as the measurement process itself is subject to the very OS-jitter you are trying to mitigate. Your architecture must include a concrete, external verification strategy. How will you objectively measure and prove the physical simultaneity of events across multiple distributed nodes to validate your $< 100 \mu\text{s}$ claim? Your design must detail the specific mechanisms and required external tools to provide this irrefutable proof.

Next Step:

Copy this text and paste it into your AI assistant with the Phase 1 Prompt:

"Review these requirements. Point out missing and inconsistent requirements. Extrapolate implementation implications. Consider hardware and software constraints and technology limitation. Extract open questions that guide the architecture. Number every requirement (F-1, NF-1, U-1)."